Group 105 - Laurier Lefebvre, Owen MacDonald, Johnny Weng Lin, Ayush Srivastava

Relational Schema

Users(emailAddress, password)

RegularUser(FOREIGN KEY (emailAddress) REFERENCES Users)

Admin(FOREIGN KEY (emailAddress) REFERENCES Users)

Genre(genreName)

favourite_genre(FOREIGN KEY (emailAddress) REFERENCES RegularUser, FOREIGN KEY (genreName) REFERENCES Genre)

Artist(artistId, name)

favourite_artist(FOREIGN KEY (emailAddress) REFERENCES RegularUser, FOREIGN KEY (artistId) REFERENCES Artist)

artist_genre(FOREIGN KEY (genreName) REFERENCES Genre, FOREIGN KEY (artistId) REFERENCES Artist)

Venue(address, venueName, capacity)

venues_near_me(FOREIGN KEY (emailAddress) REFERENCES RegularUser, FOREIGN KEY (address) REFERENCES Venue)

venue_genre(FOREIGN KEY (genreName) REFERENCES Genre, FOREIGN KEY (address) REFERENCES Venue)

concert(FOREIGN KEY (artistId) REFERENCES Artist, FOREIGN KEY (address) REFERENCES Venue, concertDate)

Tickets(FOREIGN KEY (artistId) REFERENCES Artist, FOREIGN KEY (address) REFERENCES Venue, FOREIGN KEY (concertDate) REFERENCES concert, section, seatNumber, price)

online_purchase(FOREIGN KEY (artistId, address, section, seatNumber, concertDate) REFERENCES Tickets, FOREIGN KEY (emailAddress) REFERENCES RegularUser)

in-person_purchase(FOREIGN KEY (artistId, address, section, seatNumber, concertDate) REFERENCES Tickets, FOREIGN KEY (emailAddress) REFERENCES Admin)

scan(FOREIGN KEY (artistId, address, section, seatNumber, concertDate) REFERENCES Tickets, FOREIGN KEY (emailAddress) REFERENCES Admin)

Pending Constraints

- Participation constraints for artist_genre and venues_near_me
- Non-negative value for venue capacities
- When a concert is created, concertDate is a date in the future

SQL Queries

Query 1:

```
SELECT DISTINCT
   fg.emailAddress,
   a.name AS artistName,
   fg.genreName
FROM
   favourite_genre fg
JOIN
   artist_genre ag
   ON fg.genreName = ag.genreName
JOIN
   Artist a
   ON ag.artistId = a.artistId
ORDER BY
   fg.emailAddress,
   a.name;
```

This query finds artists whose genres math the favourite genres of users. It joins the favourite genres of users with the genres associated with artists and returns the artists whose genres match the preferences of users.



```
cs421g105@winter2026-comp421:~$ db2 -tvf query_tests.sql
SELECT DISTINCT fg.emailAddress, a.name AS artistName, fg.genreName FROM favourite_genre fg JOIN artist_genre ag ON fg.genreName = ag.genreName JOIN Artist a ON ag.artistId = a.artistId ORDER BY fg.emailAddress, a.name

EMAILADDRESS                   ARTISTNAME                     GENRENAME
------------------------------ ------------------------------ --------------------------------------------------
bea.trice@bee.com              Fleetwood Mac                  70s
frodo.baggins@hobbit.ca        Avicii                         electronic
frodo.baggins@hobbit.ca        Dire Straits                   rock
frodo.baggins@hobbit.ca        Fleetwood Mac                  rock
frodo.baggins@hobbit.ca        Queen                          rock
frodo.baggins@hobbit.ca        The Beatles                    rock
harry.potter@hogwarts.com      Chopin                         classical
harry.potter@hogwarts.com      Mozart                         classical
mike.ikel23@mikeike.com        Avicii                         electronic
mrandmssmith@hollywood.com     Alphaville                     pop
mrandmssmith@hollywood.com     Madonna                        pop
mrandmssmith@hollywood.com     Mariah Carey                   pop

  12 record(s) selected.
```

Query 2:

```
SELECT DISTINCT
   fg.emailAddress,
   v.venueName,
   v.address,
   c.concertDate,
   a.name AS artistName,
   fg.genreName
FROM
   favourite_genre fg
JOIN
   artist_genre ag
   ON fg.genreName = ag.genreName
JOIN
   Artist a
   ON ag.artistId = a.artistId
JOIN
   concert c
   ON a.artistId = c.artistId
JOIN
   Venue v
   ON c.address = v.address
ORDER BY
   fg.emailAddress,
   c.concertDate,
   v.venueName;
```

This query returns concerts and venues featuring artists whose genres match the favourite genres of users. It links user preferences to artists, concerts, and venues to identify concerts that match those musical preferences.

```
SELECT DISTINCT fg.emailAddress, v.venueName, v.address, c.concertDate, a.name AS artistName, fg.genreName FROM favourite_genre fg JOIN artist_genre ag ON fg.genreName = ag.genreName JOIN Artist a ON ag.artistId = a.artistId JOI
N concert c ON a.artistId = c.artistId JOIN Venue v ON c.address = v.address ORDER BY fg.emailAddress, c.concertDate, v.venueName

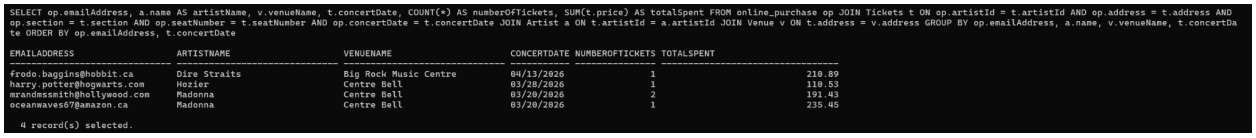
EMAILADDRESS                   VENUENAME                      ADDRESS
                                                                                                  CONCERTDATE ARTISTNAME                    GENRENAME
------------------------------ ------------------------------ ------------------------------------------------------------ ---------- ------------------------------ ------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------
bea.trice@bee.com              Big Rock Music Centre          456 South Mountain Boulevard, Vancouver, Canada
                                                                                                  05/12/2026 Fleetwood Mac                  70s
frodo.baggins@hobbit.ca        Sun Music                      164 Sunnyside Road, Ottawa, Canada
                                                                                                  03/27/2026 Avicii                         electronic
frodo.baggins@hobbit.ca        Big Rock Music Centre          456 South Mountain Boulevard, Vancouver, Canada
                                                                                                  04/13/2026 Dire Straits                   rock
frodo.baggins@hobbit.ca        Big Rock Music Centre          456 South Mountain Boulevard, Vancouver, Canada
                                                                                                  05/12/2026 Fleetwood Mac                  rock
frodo.baggins@hobbit.ca        Sun Music                      164 Sunnyside Road, Ottawa, Canada
                                                                                                  12/12/2027 Avicii                         electronic
harry.potter@hogwarts.com      Mouton Concert Hall            7506 Rue Mouton, Quebec, Canada
                                                                                                  06/23/2026 Mozart                         classical
mike.ike123@mikeike.com        Sun Music                      164 Sunnyside Road, Ottawa, Canada
                                                                                                  03/27/2026 Avicii                         electronic
mike.ike123@mikeike.com        Sun Music                      164 Sunnyside Road, Ottawa, Canada
                                                                                                  12/12/2027 Avicii                         electronic
mrandmssmith@hollywood.com     Centre Bell                    1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                                                                                  03/20/2026 Madonna                        pop

  9 record(s) selected.
```

Query 3:

```
SELECT
   op.emailAddress,
   a.name AS artistName,
   v.venueName,
   t.concertDate,
   COUNT(*) AS numberOfTickets,
   SUM(t.price) AS totalSpent
FROM
   online_purchase op
JOIN
   Tickets t
   ON (
      op.artistId = t.artistId
      AND op.address = t.address
      AND op.section = t.section
      AND op.seatNumber = t.seatNumber
      AND op.concertDate = t.concertDate
   )
JOIN
   Artist a
   ON t.artistId = a.artistId
JOIN
   Venue v
   ON t.address = v.address
GROUP BY
   op.emailAddress,
   a.name,
   v.venueName,
   t.concertDate
ORDER BY
   op.emailAddress,
   t.concertDate;
```

This query summarize the purchased online tickets. It shows for each user and each concert they purchased tickets for, the number of tickets bought and the total amount spent. It provides analytics for user to summarize their purchase history which would be valuable to the user.



```
SELECT op.emailAddress, a.name AS artistName, v.venueName, t.concertDate, COUNT(*) AS numberOfTickets, SUM(t.price) AS totalSpent FROM online_purchase op JOIN Tickets t ON op.artistId = t.artistId AND op.address = t.address AND
op.section = t.section AND op.seatNumber = t.seatNumber AND op.concertDate = t.concertDate JOIN Artist a ON t.artistId = a.artistId JOIN Venue v ON t.address = v.address GROUP BY op.emailAddress, a.name, v.venueName, t.concertDa
te ORDER BY op.emailAddress, t.concertDate

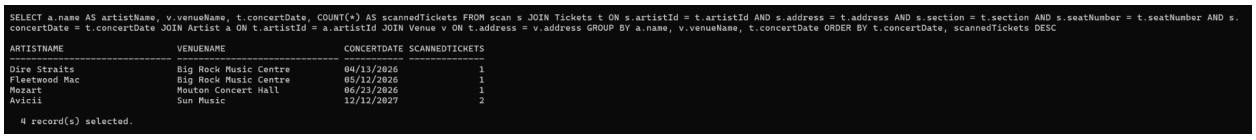
EMAILADDRESS                 ARTISTNAME                   VENUENAME                    CONCERTDATE NUMBEROFTICKETS TOTALSPENT
---------------------------- ---------------------------- ---------------------------- ----------- --------------- ----------------------------------
frodo.baggins@hobbit.ca      Dire Straits                 Big Rock Music Centre        04/13/2026              1                             210.89
harry.potter@hogwarts.com    Hozier                       Centre Bell                  03/28/2026              1                             110.53
mrandmssmith@hollywood.com   Madonna                      Centre Bell                  03/20/2026              2                             191.43
oceanwaves67@amazon.ca       Madonna                      Centre Bell                  03/20/2026              1                             235.45

  4 record(s) selected.
```

Query 4:

```
SELECT
   a.name AS artistName,
   v.venueName,
   t.concertDate,
   COUNT(*) AS scannedTickets
FROM
   scan s
JOIN
   Tickets t
   ON (
      s.artistId = t.artistId
      AND s.address = t.address
      AND s.section = t.section
      AND s.seatNumber = t.seatNumber
      AND s.concertDate = t.concertDate
   )
JOIN
   Artist a
   ON t.artistId = a.artistId
JOIN
   Venue v
   ON t.address = v.address
GROUP BY
   a.name,
   v.venueName,
   t.concertDate
ORDER BY
   t.concertDate,
   scannedTickets DESC;
```

This query counts the number of tickets that were scanned for entry at each concert. It represents actual attendance based on ticket verification. It provides information for admins or organizers of the event such that they can verify the number of attendance for the concerts.



```
SELECT a.name AS artistName, v.venueName, t.concertDate, COUNT(*) AS scannedTickets FROM scan s JOIN Tickets t ON s.artistId = t.artistId AND s.address = t.address AND s.section = t.section AND s.seatNumber = t.seatNumber AND s.
concertDate = t.concertDate JOIN Artist a ON t.artistId = a.artistId JOIN Venue v ON t.address = v.address GROUP BY a.name, v.venueName, t.concertDate ORDER BY t.concertDate, scannedTickets DESC

ARTISTNAME                     VENUENAME                      CONCERTDATE SCANNEDTICKETS
------------------------------ ------------------------------ ----------- --------------
Dire Straits                   Big Rock Music Centre          04/13/2026              1
Fleetwood Mac                  Big Rock Music Centre          05/12/2026              1
Mozart                         Mouton Concert Hall            06/23/2026              1
Avicii                         Sun Music                      12/12/2027              2

  4 record(s) selected.
```

Query 5:

```
SELECT
   a.name AS artistName,
   v.venueName,
   t.concertDate,
   COUNT(*) AS unsoldTickets
FROM
   Tickets t
JOIN
   Artist a
   ON t.artistId = a.artistId
JOIN
   Venue v
   ON t.address = v.address
LEFT JOIN
   online_purchase op
   ON (
      t.artistId = op.artistId
      AND t.address = op.address
      AND t.section = op.section
      AND t.seatNumber = op.seatNumber
      AND t.concertDate = op.concertDate
   )
WHERE
   op.emailAddress IS NULL
GROUP BY
   a.name,
   v.venueName,
   t.concertDate
ORDER BY
   t.concertDate,
   unsoldTickets DESC;
```

This query identifies concerts with unsold tickets and then proceeds to also count the tickets that exist in the ticket inventory but have not been purchased online. It provides analytics on how well each concerts are sold with a focus on online purchases.

```
SELECT a.name AS artistName, v.venueName, t.concertDate, COUNT(*) AS unsoldTickets FROM Tickets t JOIN Artist a ON t.artistId = a.artistId JOIN Venue v ON t.address = v.address LEFT JOIN online_purchase op ON t.artistId = op.art
istId AND t.address = op.address AND t.section = op.section AND t.seatNumber = op.seatNumber AND t.concertDate = op.concertDate WHERE op.emailAddress IS NULL GROUP BY a.name, v.venueName, t.concertDate ORDER BY t.concertDate, un
soldTickets DESC

ARTISTNAME                     VENUENAME                      CONCERTDATE UNSOLDTICKETS
------------------------------ ------------------------------ ----------- -------------
Madonna                        Centre Bell                    03/20/2026             10
Hozier                         Centre Bell                    03/27/2026             12
Avicii                         Sun Music                      03/27/2026              9
Hozier                         Centre Bell                    03/28/2026             12
Hozier                         Big Rock Music Centre          04/01/2026              9
Dire Straits                   Big Rock Music Centre          04/13/2026             18
Fleetwood Mac                  Big Rock Music Centre          05/12/2026              9
Mozart                         Mouton Concert Hall            06/23/2026             13
Avicii                         Sun Music                      12/12/2027             15
```

SQL Modifications

Modification 1

```
UPDATE Tickets t
SET price = price * 1.10
WHERE t.address IN (
   SELECT v.address
   FROM Venue v
   WHERE v.venueName = 'Centre Bell'
);
```

This modification increases the price of all tickets for concerts held at **Centre Bell** by **10%**. In many real-world situations, ticket prices for events at a particular venue may be adjusted due to demand, operational costs, or promotional pricing changes. This query updates multiple rows in the tickets table at once by first identifying the address associated with Centre Bell in the Venue table, and then applying a 10% price increase to every ticket associated with that venue.

```
cs421g105@winter2026-comp421:~$ db2 -tvf modifications.sql
SELECT * FROM Tickets WHERE address IN ( SELECT address FROM Venue WHERE venueName = 'Centre Bell' )

ARTISTID    ADDRESS
                                   CONCERTDATE SECTION              SEATNUMBER  PRICE
----------- ---------------------------------------------------------------------------------------------------
------------------------------------ ---------- -------------------- ----------- ------------
      10001 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                   03/20/2026  202                             13       110.53
      10001 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                   03/20/2026  310                             30        80.90
      10001 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                   03/20/2026  General Admission               12       235.45
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                   03/28/2026  202                             13       110.53
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                   03/28/2026  202                             12       110.53
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                   03/27/2026  123                             25       298.03
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                   03/27/2026  124                             23       294.21
      10001 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                   03/20/2026  211                             20       176.39
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                   03/28/2026  322                              6       126.55
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                   03/28/2026  301                             13       108.34
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                   03/27/2026  119                             22       200.51
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                   03/27/2026  414                              7        59.32
      10001 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                   03/20/2026  219                             19       169.83
      10001 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                   03/20/2026  413                             16        97.17
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                   03/28/2026  224                             30       187.34
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                   03/28/2026  315                             10       147.89
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                   03/28/2026  223                             21       185.61
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                   03/28/2026  428                             30        64.87
      10001 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                   03/20/2026  327                             10       105.99
      10001 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                   03/20/2026  224                              1       163.77
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                   03/27/2026  109                              3       232.14
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                   03/27/2026  330                             26       127.75
      10001 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                   03/20/2026  208                             14       150.58
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                   03/27/2026  230                             20       166.51
      10001 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
```

```
UPDATE Tickets t SET price = price * 1.10 WHERE t.address IN ( SELECT v.address FROM Venue v WHERE v.venueName = 'Centre Bell' )
DB20000I  The SQL command completed successfully.

SELECT * FROM Tickets WHERE address IN ( SELECT address FROM Venue WHERE venueName = 'Centre Bell' )

ARTISTID    ADDRESS
                                    CONCERTDATE SECTION             SEATNUMBER  PRICE
----------- ------------------------------------------------------------------------------------------------------------------------------------
------------------------------------ ---------- ------------------- ----------- ------------
      10001 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                    03/20/2026  202                          13       121.58
      10001 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                    03/20/2026  310                          30        88.99
      10001 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                    03/20/2026  General Admission            12       258.99
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                    03/28/2026  202                          13       121.58
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                    03/28/2026  202                          12       121.58
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                    03/27/2026  123                          25       327.83
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                    03/27/2026  124                          23       323.63
      10001 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                    03/20/2026  211                          20       194.02
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                    03/28/2026  322                           6       139.20
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                    03/28/2026  301                          13       119.17
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                    03/27/2026  119                          22       220.56
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                    03/27/2026  414                           7        65.25
      10001 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                    03/20/2026  219                          19       186.81
      10001 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                    03/20/2026  413                          16       106.88
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                    03/28/2026  224                          30       206.07
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                    03/28/2026  315                          10       162.67
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                    03/28/2026  223                          21       204.17
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                    03/28/2026  428                          30        71.35
      10001 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                    03/20/2026  327                          10       116.58
      10001 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                    03/20/2026  224                           1       180.14
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                    03/27/2026  109                           3       255.35
        416 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                    03/27/2026  330                          26       140.52
      10001 1909 Avenue des Canadien-de-Montreal, Montreal, Canada
                                    03/20/2026  208                          14       165.63
```

Modification 2

```
DELETE FROM online_purchase
WHERE address IN
(
   SELECT address
   FROM Venue
   WHERE venueName = 'Centre Bell'
)
AND concertDate = DATE('2026-03-20');
```

This modification deletes all online purchase records for the concert taking place at Centre Bell on March 20,2026. This models a scenario where a concert was cancelled, where all ticket purchases associated with the event must be removed from the system. The query first retrieves the address of Centre Bell from the Venue table and then deletes all rows in the online purchase

table that match both that venue and the specified concert date. This modification affects multiple rows and represents a realistic administrative update to the database.

```
SELECT * FROM online_purchase WHERE address IN ( SELECT address FROM Venue WHERE venueName = 'Centre Bell' ) AND concertDate = DATE('2026-03-20')

ARTISTID    ADDRESS                                                
```

```
db2 => db2 => db2 (cont.) => INSERT INTO LargeVenues (address, venueName, capacity) VALUES ('200 King Street, Calgary, Canada', 'Prairie Arena'
, 12000)
DB21034E  The command was processed as an SQL statement because it was not a
valid Command Line Processor command.  During SQL processing it returned:
SQL0803N  One or more values in the INSERT statement, UPDATE statement, or
foreign key update caused by a DELETE statement are not valid because the
primary key, unique constraint or unique index identified by "1" constrains
table "CS421G105.VENUE" from having duplicate values for the index key.
SQLSTATE=23505
```

The insert succeeds because this view is on a single base table (Venue) with no joins, aggregation, or DISTINCT, so DB2 treats it as updatable. The row goes directly into Venue, and since capacity 12000 > 5000, it's visible through the view. Without WITH CHECK OPTION, even a row with capacity <= 5000 would be accepted into Venue but just wouldn't appear in the view.

View 2

This view summarizes each concert by showing the artist, venue, date, average ticket price, and total tickets sold — joining Concert, Artist, Venue, and Tickets with aggregation.

```
CREATE VIEW ConcertSummary (artistName, venueName, concertDate, avgTicketPrice,
totalTickets) AS
SELECT a.name, v.venueName, c.concertDate,
      AVG(t.price), COUNT(*)
FROM concert c
JOIN Artist a ON c.artistId = a.artistId
JOIN Venue v ON c.address = v.address
JOIN Tickets t ON c.artistId = t.artistId
   AND c.address = t.address
   AND c.concertDate = t.concertDate
GROUP BY a.name, v.venueName, c.concertDate;
```

```
db2 => db2 => db2 => db2 => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.
) => db2 (cont.) => CREATE VIEW ConcertSummary (artistName, venueName, concertDate, avgTicketPrice, totalTickets) AS SELECT a.name, v.venueName
, c.concertDate, AVG(t.price), COUNT(*) FROM concert c JOIN Artist a ON c.artistId = a.artistId JOIN Venue v ON c.address = v.address JOIN Tick
ets t ON c.artistId = t.artistId AND c.address = t.address AND c.concertDate = t.concertDate GROUP BY a.name, v.venueName, c.concertDate
DB21034E  The command was processed as an SQL statement because it was not a
valid Command Line Processor command.  During SQL processing it returned:
SQL0601N  The name of the object to be created is identical to the existing
name "CS421G105.CONCERTSUMMARY" of type "VIEW".  SQLSTATE=42710
```

```
db2 => db2 => db2 => SELECT * FROM ConcertSummary FETCH FIRST 5 ROWS ONLY

ARTISTNAME                     VENUENAME                     CONCERTDATE AVGTICKETPRICE                    TOTALTICKETS
------------------------------ ----------------------------- ----------- --------------------------------- ------------
Madonna                        Centre Bell                   03/20/2026        183.8984615384615384615384           13
Avicii                         Sun Music                     03/27/2026        150.7844444444444444444444            9
Hozier                         Centre Bell                   03/27/2026        203.1850000000000000000000           12
Hozier                         Centre Bell                   03/28/2026        140.2223076923076923076923           13
Hozier                         Big Rock Music Centre         04/01/2026        136.1555555555555555555555            9
```

```
db2 => db2 => db2 => db2 (cont.) => INSERT INTO ConcertSummary (artistName, venueName, concertDate, avgTicketPrice, totalTickets) VALUES ('Test
 Artist', 'Test Venue', '2026-12-01', 100.00, 5)
DB21034E  The command was processed as an SQL statement because it was not a
valid Command Line Processor command.  During SQL processing it returned:
SQL0150N  The target fullselect, view, typed table, materialized query table,
range-clustered table, or staging table in the INSERT, DELETE, UPDATE, MERGE,
or TRUNCATE statement is a target for which the requested operation is not
permitted.  SQLSTATE=42807

db2 => cs421g105@winter2026-comp421:~$
```

The insert fails with SQL0150N (SQLSTATE 42807). DB2 considers this view read-only because it contains joins across multiple tables, aggregate functions (AVG, COUNT), and GROUP BY. DB2 can't map a single insert into the correct changes across four underlying tables, so the operation is not permitted.

Check Constraints

Check 1: Validate the concert dates

ALTER TABLE concert
ADD CONSTRAINT validateDate CHECK (concertDate >= ‘2026-03-11’);

This constraint ensures that when a concert tuple is created, the concert’s date is in the future. This is because there is no point in creating a concert happening in the past. This check constraint is named validateDate so it can be easily updated every day to ensure new concert tuples have a valid date.

Running this command on the database.

```
cs421g105@winter2026-comp421:~$ db2 "ALTER TABLE concert ADD CONSTRAINT validateDate CHECK (concertDate >= '2026-03-11');"
DB20000I  The SQL command completed successfully.
```

Inserting a record that violates this constraint.

```
cs421g105@winter2026-comp421:~$ db2 "INSERT INTO concert (artistId, address, concertDate) VALUES (10001, '1909 Avenue des Canadien-de-Montreal, Montreal, Canada', '2026-03-10');"
DB21034E  The command was processed as an SQL statement because it was not a
valid Command Line Processor command.  During SQL processing it returned:
SQL0545N  The requested operation is not allowed because a row does not
satisfy the check constraint "CS421G105.CONCERT.VALIDATEDATE".  SQLSTATE=23513
```

Check 2: Validate the venues’ capacities

ALTER TABLE Venue
ADD CHECK (capacity >= 0);

This constraint makes sure that each venue has a non-negative capacity. This is because having a negative capacity has no meaning. There cannot be negative people in a venue. It is unnecessary to name this constraint because it shouldn’t be modified.

Running this command on the database.

```
cs421g105@winter2026-comp421:~$ db2 "ALTER TABLE Venue ADD CHECK (capacity >= 0);"
DB20000I  The SQL command completed successfully.
```

Inserting a record that violates this constraint.

```
cs421g105@winter2026-comp421:~$ db2 "INSERT INTO Venue (address, venueName, capacity) VALUES ('1310 Rue Sherbrooke, Montreal, Canada', 'Con Straint Centre', -200);"
DB21034E  The command was processed as an SQL statement because it was not a
valid Command Line Processor command.  During SQL processing it returned:
SQL0545N  The requested operation is not allowed because a row does not
satisfy the check constraint "CS421G105.VENUE.SQL260311090918150".
SQLSTATE=23513
```

Creativity

Real data sets:

- The table with meaningful data is the Tickets table. This table is composed of 12 manually written tuples (mainly the ones referenced by other tables) and 100 randomly generated tuples. Those 100 tuples were randomly generated and formatted by a python script (see attachment: tupleGenerator.py).
- This script uses a while loop that iterates NUM_TICKETS times, in this case, 100 times. The loop starts by choosing a random concert tuple from the concert table (which needs to be manually added at the start of the script). It then chooses a random section label from a pool of pre-defined labels (also at the start of the script), and finally chooses a random seat number. If this combination of random elements is already used, this iteration of the loop is cancelled and the ticket tuple is not added to the pool of generated tickets. If it hasn't been used, its usage is recorded and a random price is generated within a certain range depending on the section of the venue. The ticket is then added to the pool of tickets. Once there are 100 tickets, they are formatted so that the output can be copied and pasted into an INSERT statement.
- This method of generating tuples comes with a few flaws. For example the section labels are limited to the same format for every venue when realistically, it is possible that different venues have differently labelled sections. Also, realistically, some artists' concerts are generally more expensive than others because of other factors like popularity, however, this script does not represent that. Despite these minor flaws, the data is still very possible and realistic.

Advanced SQL: Window Functions

We used window functions here. The query ranks concerts by ticket revenue, tracks a running total of revenue over time, and shows the percentage each concert contributes to the overall revenue.

The window functions we used:

- RANK() OVER (ORDER BY totalRevenue DESC): gives each concert a rank based on revenue
- SUM() OVER (ORDER BY concertDate ROWS UNBOUNDED PRECEDING): cumulative revenue as concerts happen over time
- SUM() OVER (): total revenue across all concerts, used to get each concert's percentage
- We also used a CTE (WITH) to calculate per-concert revenue first before applying the window functions on top

```
WITH ConcertRevenue AS (
   SELECT a.name AS artistName,
          v.venueName,
          c.concertDate,
          COUNT(*) AS ticketsSold,
          SUM(t.price) AS totalRevenue
   FROM concert c
   JOIN Artist a ON c.artistId = a.artistId
   JOIN Venue v ON c.address = v.address
   JOIN Tickets t ON c.artistId = t.artistId
      AND c.address = t.address
      AND c.concertDate = t.concertDate
   GROUP BY a.name, v.venueName, c.concertDate
)
SELECT artistName,
       venueName,
       concertDate,
       ticketsSold,
       totalRevenue,
       RANK() OVER (ORDER BY totalRevenue DESC) AS revenueRank,
       SUM(totalRevenue) OVER (ORDER BY concertDate
          ROWS UNBOUNDED PRECEDING) AS cumulativeRevenue,
       DECIMAL(totalRevenue * 100.0
          / SUM(totalRevenue) OVER (), 5, 2) AS pctOfTotal
FROM ConcertRevenue
ORDER BY revenueRank;
```

```
db2 => db2 => db2 => db2 => db2 => db2 => db2 => db2 => db2 => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) =>
 db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => d
b2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => WITH ConcertRevenue AS ( SELECT a.name AS artistName, v.venueNa
me, c.concertDate, COUNT(*) AS ticketsSold, SUM(t.price) AS totalRevenue FROM concert c JOIN Artist a ON c.artistId = a.artistId JOIN Venue v ON c.address = v.address
JOIN Tickets t ON c.artistId = t.artistId AND c.address = t.address AND c.concertDate = t.concertDate GROUP BY a.name, v.venueName, c.concertDate ) SELECT artistName,
venueName, concertDate, ticketsSold, totalRevenue, RANK() OVER (ORDER BY totalRevenue DESC) AS revenueRank, SUM(totalRevenue) OVER (ORDER BY concertDate ROWS UNBOUNDED
 PRECEDING) AS cumulativeRevenue, DECIMAL(totalRevenue * 100.0 / SUM(totalRevenue) OVER (), 5, 2) AS pctOfTotal FROM ConcertRevenue ORDER BY revenueRank

ARTISTNAME                     VENUENAME                     CONCERTDATE TICKETSSOLD TOTALREVENUE                    REVENUERANK          CUMULATIVEREVENUE
       PCTOFTOTAL
------------------------------ ------------------------------ ----------- ----------- --------------------------------- -------------------- --------------------------
------- ----------
Avicii                         Sun Music                     12/12/2027           15                           3805.25                    1                          1
8829.79      20.20
Dire Straits                   Big Rock Music Centre         04/13/2026           19                           3048.62                    2                          1
1678.30      16.10
Hozier                         Centre Bell                   03/27/2026           12                           2216.60                    3
5747.06      11.70
Madonna                        Centre Bell                   03/20/2026           13                           2173.40                    4
2173.40      11.50
Mozart                         Mouton Concert Hall           06/23/2026           13                           2055.83                    5                          1
5024.54      10.90
Hozier                         Centre Bell                   03/28/2026           13                           1657.22                    6
7404.28       8.80
Avicii                         Sun Music                     03/27/2026            9                           1357.06                    7
3530.46       7.20
Fleetwood Mac                  Big Rock Music Centre         05/12/2026            9                           1290.41                    8                          1
2968.71       6.80
Hozier                         Big Rock Music Centre         04/01/2026            9                           1225.40                    9
8629.68       6.50

  9 record(s) selected.
```

Collaboration

We set up a meeting at the start of the project period to discuss how we were going to split up the workload and to iron out any questions we had after reading the project description. Although we did not have any more meetings after the initial one, we kept constant communication in a group chat throughout the rest of the project. Laurier did steps 1-3: setting up the group account and loading the files from the project description into our home directory, modifying the relational schema based on the feedback we received from part 1 of the project, and making the create table SQL statements. He also formalized the final report and gathered all files for submission. Owen loaded data into the database (Q4), added two CHECK constraints (Q8), generated meaningful data for one table (Q9: Real data sets), and helped formalize the report. Ayush created the two views (Q7) and wrote the advanced SQL window function query for the creativity section (Q9). Johnny wrote the five advanced queries (Q5) and wrote two complex data modifications (Q6).